МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМ. ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра мережевих та інтернет технологій

СИСТЕМНЕ ПРОГРАМУВАННЯ

Лабораторна робота №1

ПОТОКИ. КЛАС THREAD. ПРІОРИТЕТИ ПОТОКІВ

Довгодька Дениса Станіславовича

Мета: ознайомитись із основами потоків процесу, класом Thread та пріоритетами потоків.

Хід роботи

Завдання 1.

Реалізуйте програму на C#, яка створює два потоки: Перший потік виводить числа від 1 до 40 з паузою 200 мс, ррругий потік виводить літери від A до Z з паузою 300 мс. Використайте методи Start(), Sleep() та властивість Name. Поясніть, як працює багатозадачність у межах одного процесу.

Створимо програмний застосунок, який реалізує задачу. Вивід в консоль відображено на рисунку 1.1.

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Priority & Background)
3 - Run Task 3 (Static Priority Test)
4 - Run Task 4 (Interactive Thread Monitor)
0 - Exit

Select task: 1
NumbersThread: 1
LettersThread: A
NumbersThread: 2
LettersThread: B
NumbersThread: 3
LettersThread: C
NumbersThread: 4
NumbersThread: 5
LettersThread: D
NumbersThread: 6
LettersThread: E
NumbersThread: 7
NumbersThread: 8
LettersThread: F
NumbersThread: 9
LettersThread: G
NumbersThread: 10
NumbersThread: 11
LettersThread: H
NumbersThread: 12
LettersThread: I
```

Рисунок 1.1 – Виведення чисел та літер різними потоками.

Як бачимо, лог виведення між різними потоками неоднорідний, що вказує на їх паралельність виконання із різними затримками в рантаймі.

Один процес може містити кілька потоків виконання, які працюють паралельно та використовують спільну пам’ять і ресурси процесу. Кожен потік виконує свою частину коду, а операційна система або середовище виконання розподіляє процесорний час між

ними через планувальник. Це може відбуватися або паралельно на багатоядерних процесорах, або по черзі із перемиканням контексту.

Такий підхід дозволяє ефективніше використовувати ресурси та пришвидшує виконання програми, але потребує синхронізації щоб уникнути конфліктів доступу до спільних даних.

Завдання 2.

Створіть програму, яка запускає три потоки: два пріоритетні (звичайні), один фоновий. Організуйте так, щоб фоновий потік виконував нескінченний цикл з повідомленням у консоль. Що відбудеться з фоновим потоком після завершення пріоритетних потоків?

Маємо наступний вивід із алгоритму (рис. 1.2)

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Priority & Background)
3 - Run Task 3 (Static Priority Test)
4 - Run Task 4 (Interactive Thread Monitor)
0 - Exit

Select task: 2
P1 started
P1: 0
P2 started
P2: 0
Background thread working...
P2: 1
P1: 1
P2: 2
P1: 2
Background thread working...
P2: 3
P1: 3
P1: 4
P2: 4
P2: 5
P1: 5
Background thread working...
P1: 6
P2: 6
P2: 7
P1: 7
Background thread working...
P1: 8
P2: 8
P1: 9
P2: 9
Background thread working...
P1 finished
P2 finished

Task finished. Press Enter to return to menu...
Background thread working...
Background thread working...
```

Рисунок 1.2 – Вивід завдання з трьома потоками

В разі завершення пріоритетних потоків, фоновий завершається за умови, що в процесі немає жодних основних потоків. У випадку вище, потік основного методу Main.

Завдання 3.

Створити багатопотокову програму, задати різні пріоритети для потоків. Кожен потік виконує підрахунок до 100 млн. Після завершення роботи програми визначте кількість ітерацій, виконаних кожним потоком, та розрахуйте відсотковий розподіл процесорного часу.

Потоків – 5 шт. (пріоритети: найвищий, найнижчий, вище середнього, середній, нижче середнього).

Чи завжди високо пріоритетний потік отримує найбільше часу ЦП? Які фактори можуть впливати на планування?

Отримаємо наступний вивід результату виконання створеного алгоритму для вирішення задачі (рис. 1.3)

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Priority & Background)
3 - Run Task 3 (Static Priority Test)
4 - Run Task 4 (Interactive Thread Monitor)
0 - Exit

Select task: 3

Task 3 results:
0: Highest | 67,35%
1: Lowest | 0,27%
2: AboveNormal | 28,15%
3: Normal | 4,23%
4: BelowNormal | 0,00%
```

Рисунок 1.3 – Результат роботи на одному ядрі потоків з різними пріоритетами

Планувальник ОС лише надає перевагу за пріоритетами, але не гарантує повне домінування.

На планування впливають:

- кількість потоків і їх пріоритети
- політика планувальника ОС
- кількість ядер процесора
- блокування (I/O, очікування ресурсів)
- перемикання контексту та навантаження системи

Завдання 4.

Високий рівень: Розробіть консольний застосунок, який:

- Дозволяє користувачу ввести кількість потоків та їхні пріоритети.
- Запускає кожен потік із власним завданням (наприклад, підрахунок до заданого числа).
- Виводить у реальному часі хід виконання кожного потоку (лічильник, стан).
- Після завершення – формує таблицю розподілу процесорного часу між потоками у %.

Виведемо результат роботи програми (рис. 1.4-1.6)

```
--- Main Menu ---
1 - Run Task 1 (Numbers & Letters)
2 - Run Task 2 (Priority & Background)
3 - Run Task 3 (Static Priority Test)
4 - Run Task 4 (Interactive Thread Monitor)
0 - Exit

Select task: 4
Threads count: 3
Thread 0 priority (lowest/below/normal/above/highest): below
Thread 1 priority (lowest/below/normal/above/highest): normal
Thread 2 priority (lowest/below/normal/above/highest): above
```

Рисунок 1.4 – Створення потоків з різними пріоритетами

```
Live:

T0 | BelowNormal | 214470832 | Done
T1 | Normal | 1344259539 | Done
T2 | AboveNormal | 80075842384 | Done
```

Рисунок 1.5 – Робота потоків в живому часі

```
Result:

T0 | BelowNormal | 0,24%
T1 | Normal | 1,84%
T2 | AboveNormal | 97,93%

Task finished. Press Enter to return to menu...
```

Рисунок 1.6 – Статистичний підсумок розподілу по потокам після завершення виконання

Висновок: У ході лабораторної роботи була досліджена багатозадачність у межах одного процесу за допомогою потоків. Створено кілька потоків з різними пріоритетами, включаючи фоновий. Показано, що високопріоритетні потоки отримують перевагу у виконанні, але не завжди більше часу ЦП, оскільки на планування впливають блокування, кількість потоків, навантаження та політика ОС. Фонові потоки продовжують виконання, поки живий хоча б один основний потік процесу. Для синхронізації використовувався метод Join(), який забезпечує очікування завершення потоків і контроль порядку виконання.